

**Mansoura University
Faculty of Computers
and Information
Department of
Information System
First Semester- 2024-
2025**

Agile Methods

Assoc.Prof: Samir Abdelrazek



Lecture 12

Object Modeling (O-O-A)

UMLs

**Assoc. Prof: Samir
Abd ElRazek**

2024



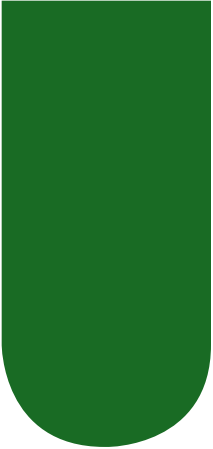
Object Modeling Using UMLs

UMLS

- Use Case Modeling
- Use Case Diagram
- Class Diagrams
- Sequence Diagrams
- State Transitions Diagrams
- Activity Diagrams
- BPM
- Case Tools

**UNIFIED
MODELING
LANGUAGE™**





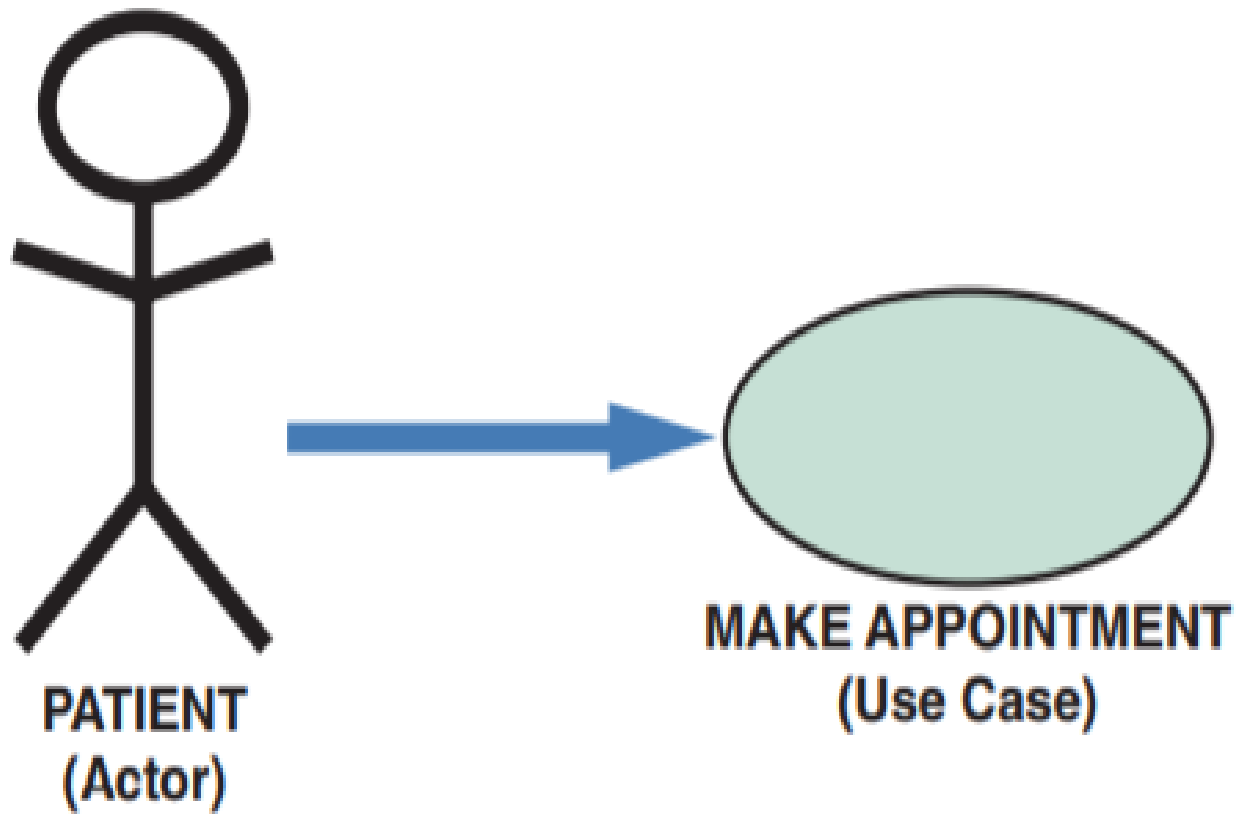
**UNIFIED
MODELING
LANGUAGE™**



UMLs

UML is a popular technique for documenting and modeling a system. UML uses a set of symbols to represent **graphically** the various components and relationships within a system.

Although the UML can be used for business process modeling and requirements modeling, it is mainly used to support O-O system analysis and to develop object models.

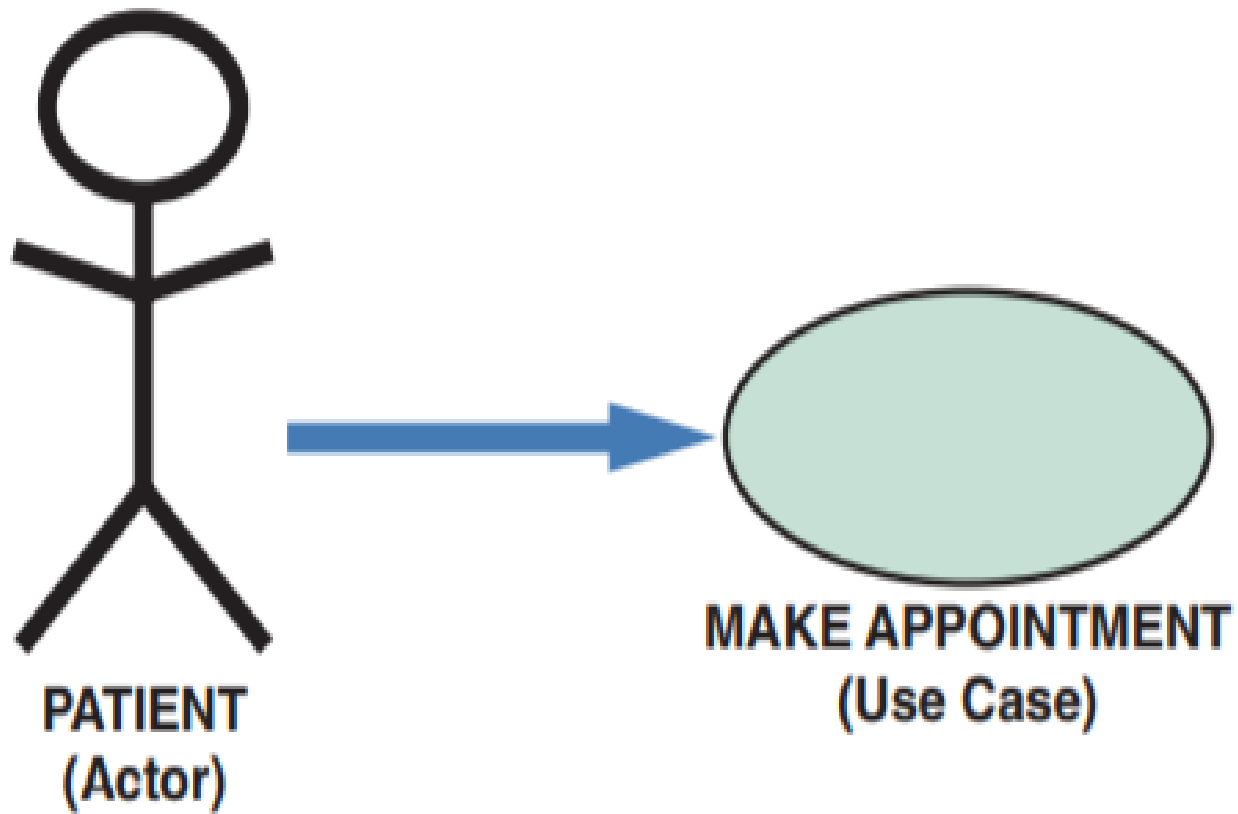


USE CASE MODELING

A use case represents the **steps** in a specific business function or process.

An external entity, called an **actor**, initiates a use case by requesting the system to perform a function or process.

For example, in a medical office system, a **PATIENT (actor)** can **MAKE APPOINTMENT (use case)**.

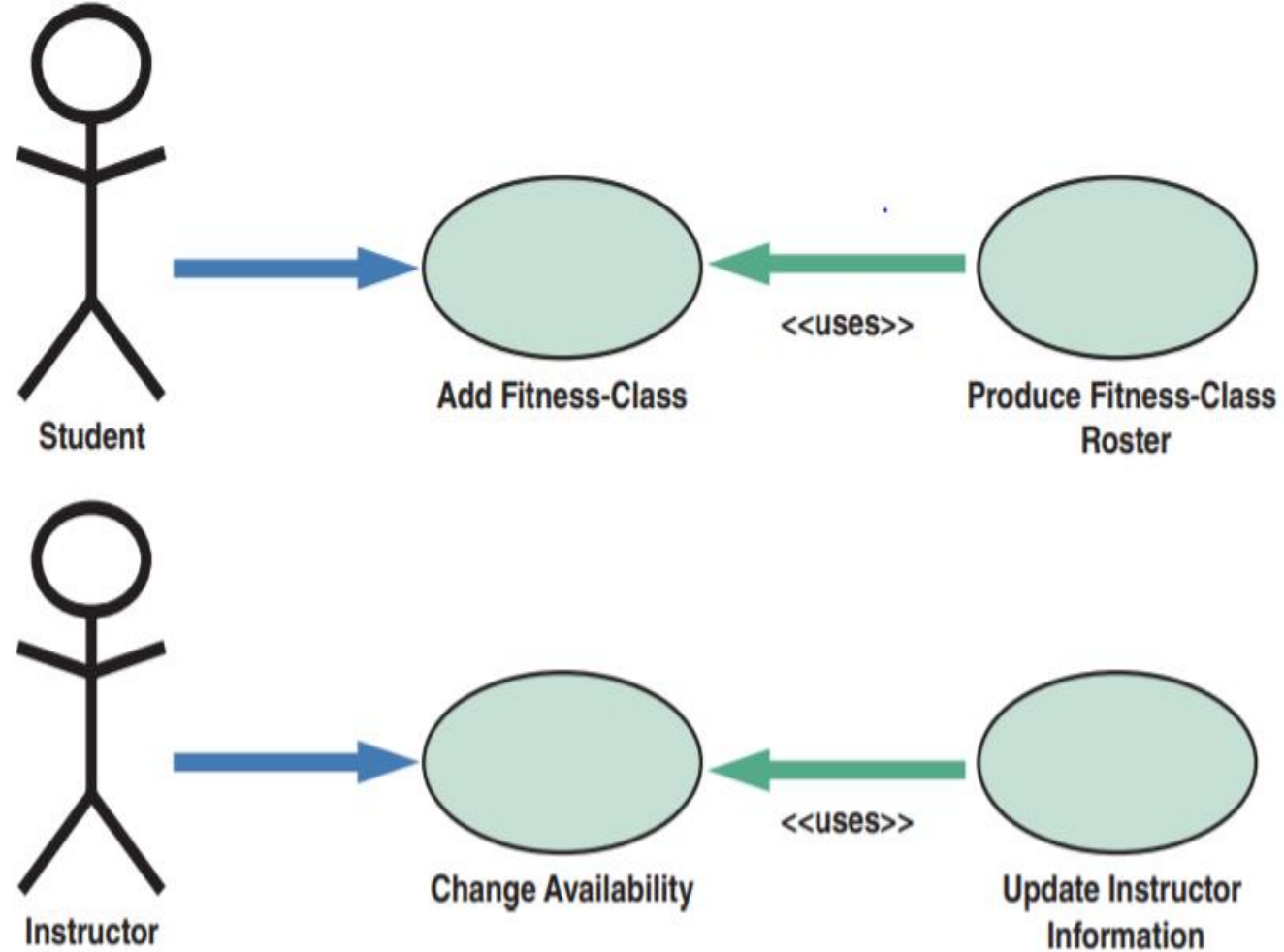


USE CASE MODELING

Use cases also can interact with other use cases.

When the outcome of one use case is incorporated by another use case, we say that the second case **uses** the first case

UML indicates the relationship with **an arrow** that points at the use case being used.



USE CASE MODELING

Student adds a fitness class, and Produce Fitness-Class Roster **uses** the results of Add Fitness-Class to generate a new fitness-class roster.

ADD NEW STUDENT Use Case



Name: Add New Student

Actor: Student/Manager

Description: Describes the process used to add a student to a fitness-class

Successful completion:

1. Manager checks FITNESS-CLASS SCHEDULE object for availability
2. Manager notifies student
3. Fitness-class is open and student pays fee
4. Manager registers student

Alternative:

1. Manager checks FITNESS-CLASS SCHEDULE object for availability
2. Fitness-class is full
3. Manager notifies student

Precondition: Student requests fitness-class

Postcondition: Student is enrolled in fitness-class and fees have been paid

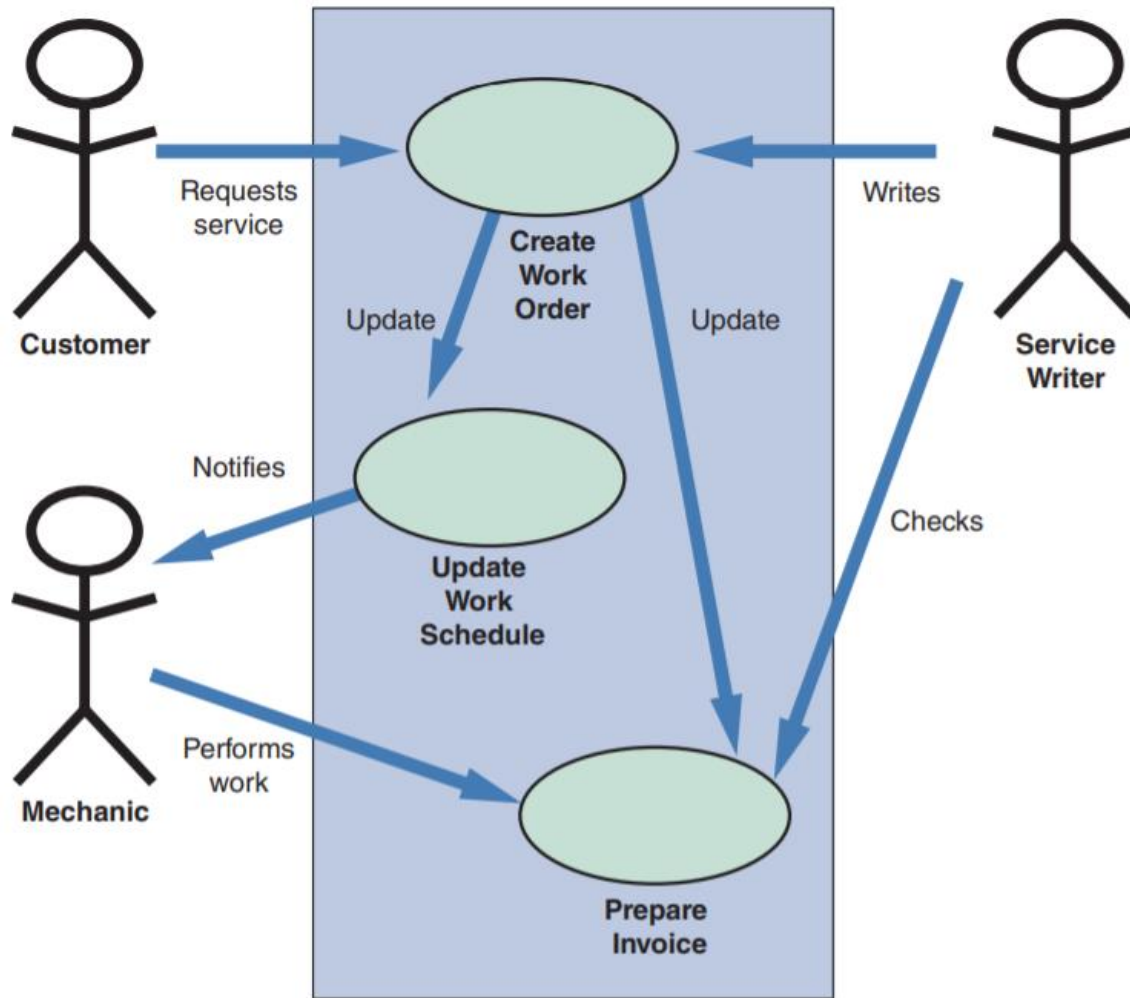
Assumptions: None

Use case description

For each use case, develop a use case description in the form of a table.

A use case description documents the name of the use case, the actor, a description of the use case, a **step-by-step** list of the tasks and actions required for successful completion.

Use Case Diagram: Auto Service Department

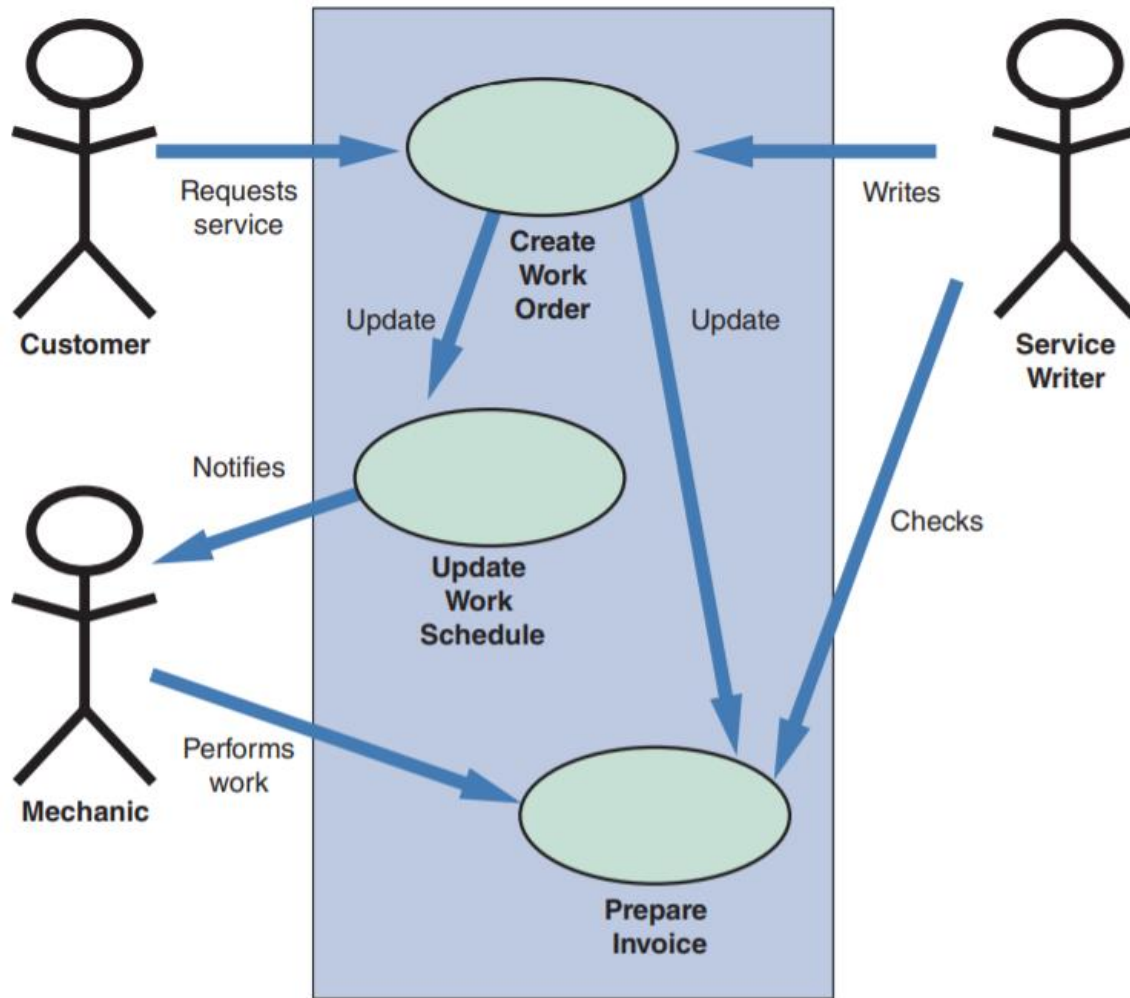


USE CASE DIAGRAMS

A use case diagram is a **visual summary of several related use cases** within a system or subsystem.

Consider a typical auto service department involves customers; service writers, who prepare work orders and invoices; and mechanics, who perform the work.

Use Case Diagram: Auto Service Department



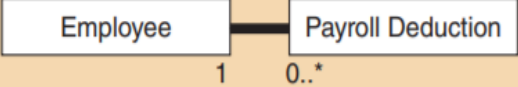
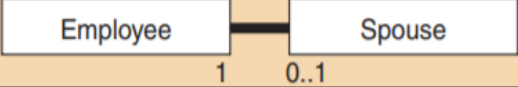
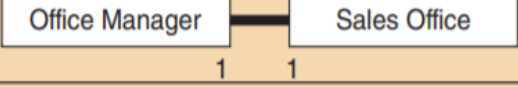
USE CASE DIAGRAMS

When a use case diagram is created, the first step is to identify the **system boundary**, which is represented by a **rectangle**.

The **system boundary** shows what is included in the system (inside the rectangle) and what is not included in the system (outside the rectangle).

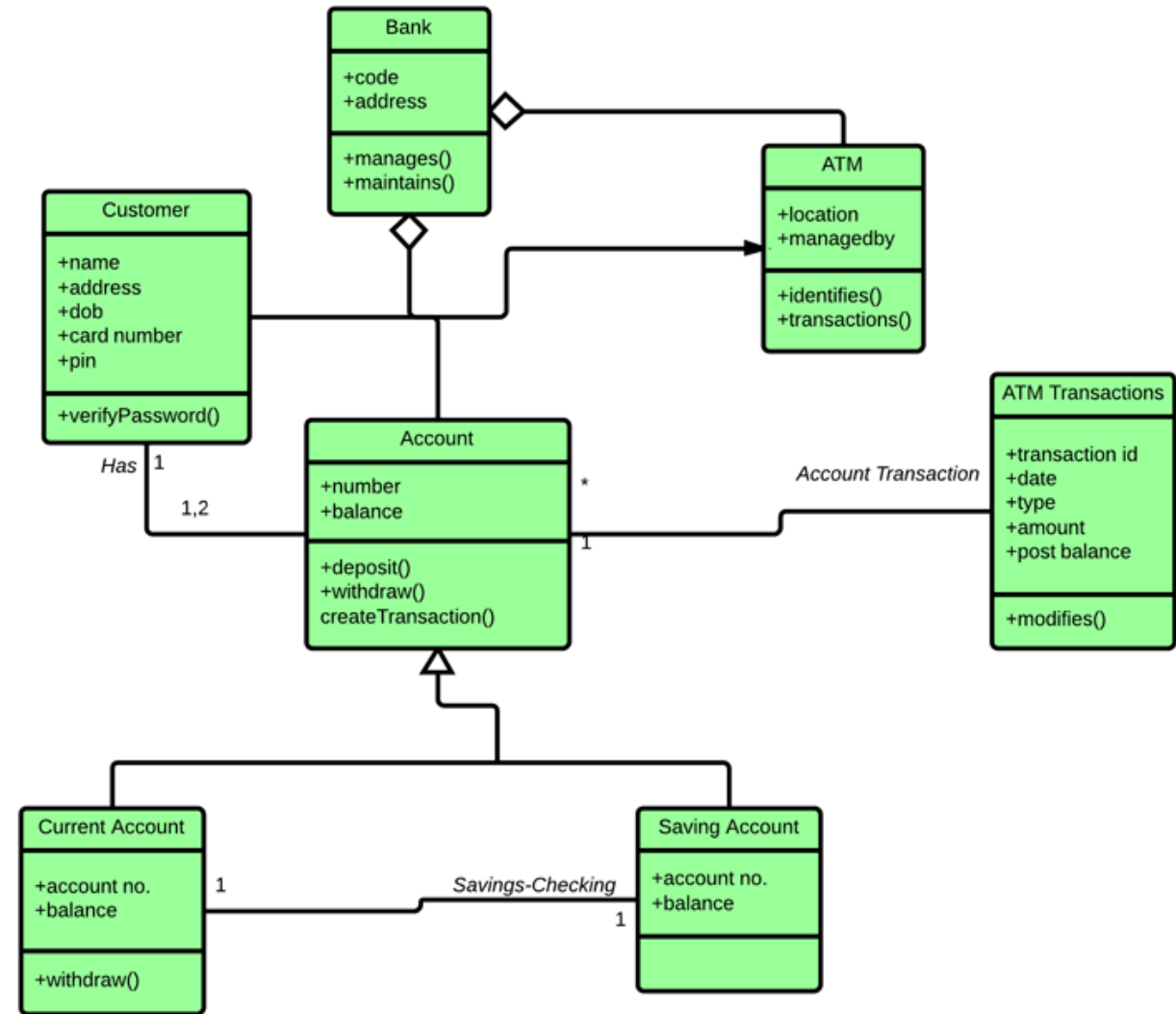
Class Diagram

- A **class diagram** shows the object classes and relationships involved in a use case. Like a DFD, a class diagram is a logical model, which evolves into a physical model and finally becomes a functioning information system.

UML Notation	Nature of the Relationship	Example		Description
0..*	Zero or many			An employee can have no payroll deductions or many deductions.
0..1	Zero or one			An employee can have no spouse or one spouse.
1	One and only one			An office manager manages one and only one office.
1..*	One or many			One order can include one or many items ordered.

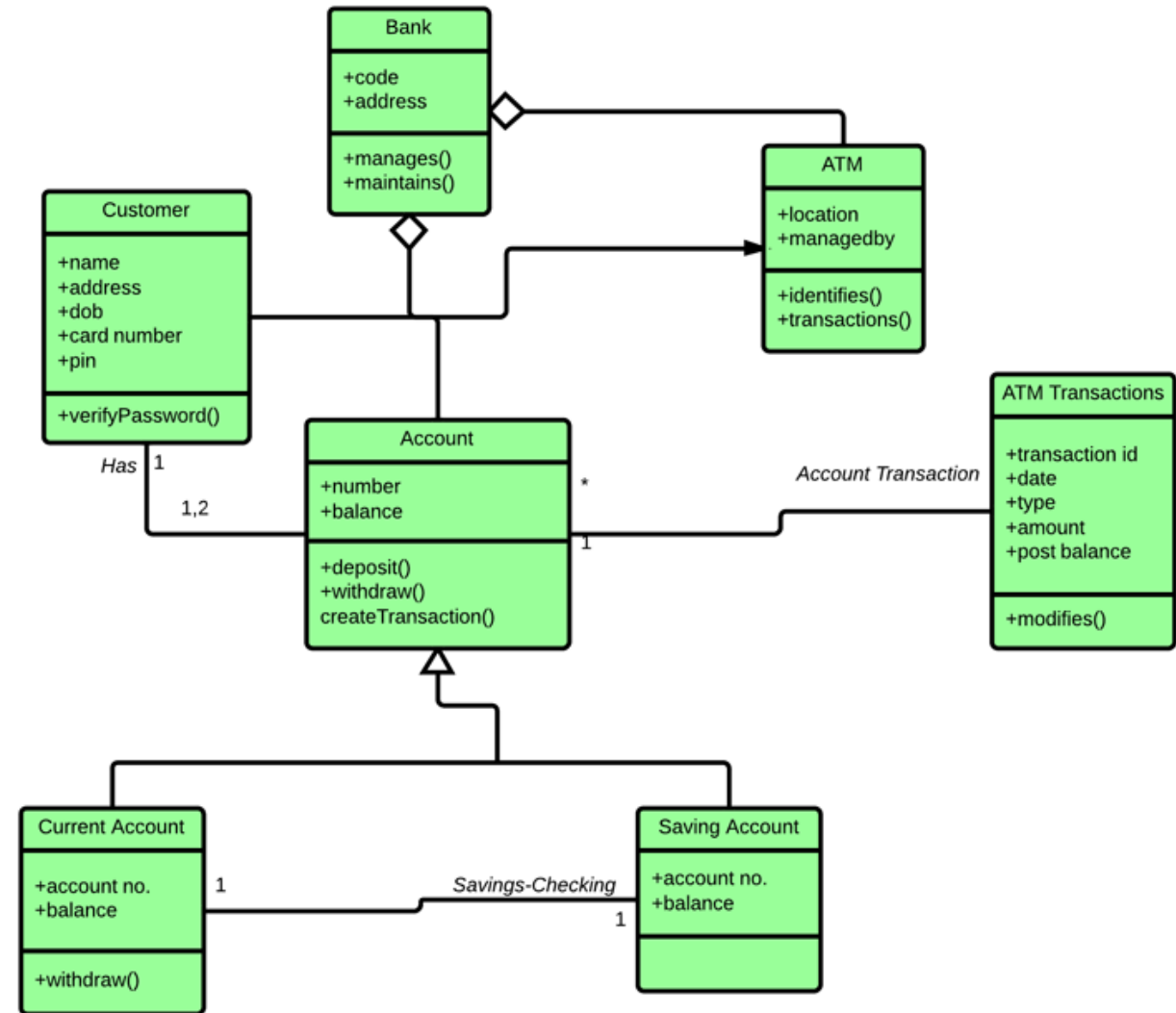
Class Diagram

- A **class diagram** shows the object classes and relationships involved in a use case. Like a DFD, a class diagram is a logical model, which evolves into a physical model and finally becomes a functioning information system.



Class Diagram

- In structured analysis, entities, data stores, and processes are transformed into data structures and program code. Similarly, class diagrams evolve into code modules, data objects, and other system components.



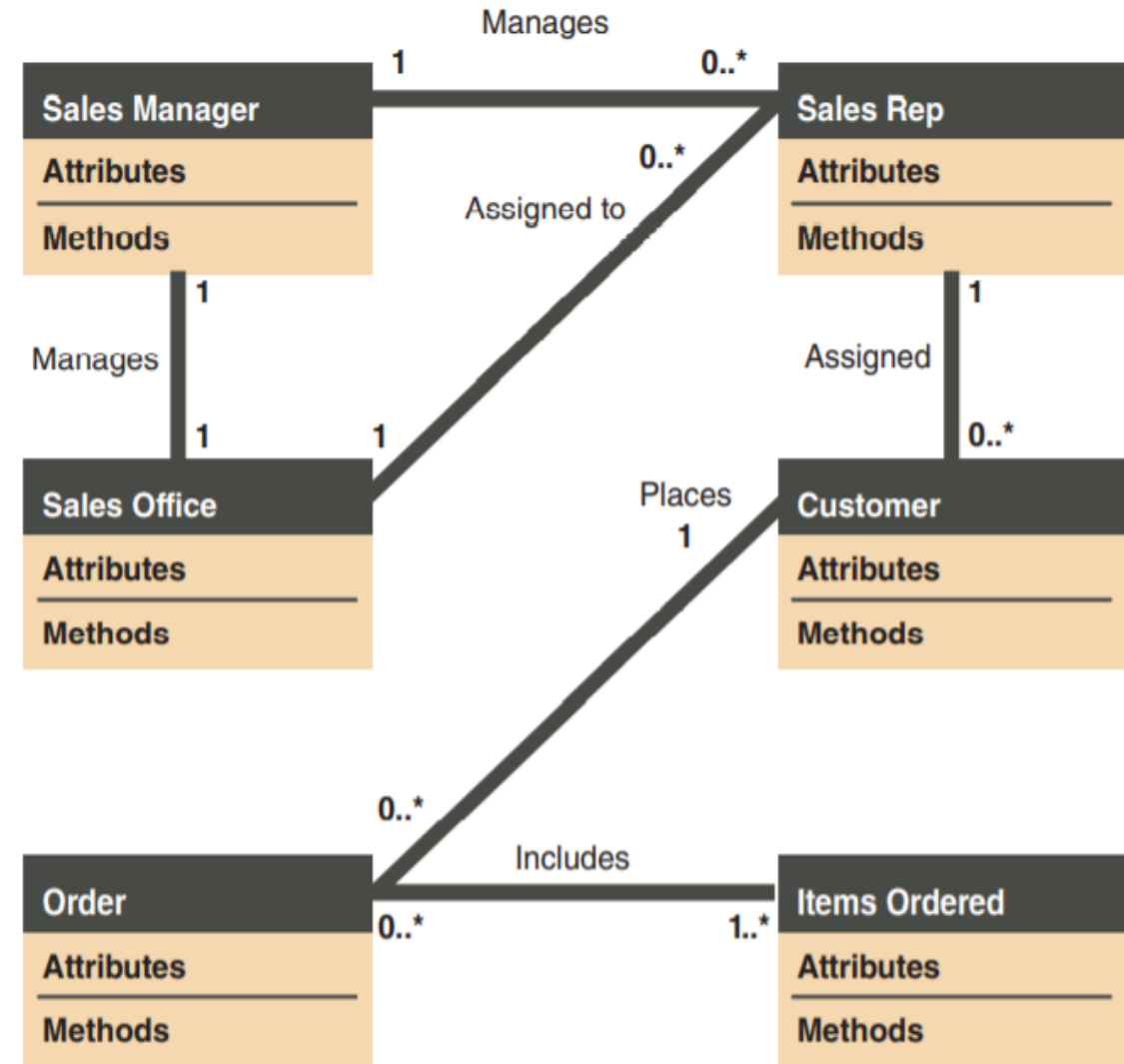
Cardinality

- The class diagram also includes a concept called **cardinality**, which describes how instances of one class relate to instances of another class.
- For example, an employee might have earned no vacation days or one vacation day or many vacation days.
- Similarly, an employee might have no spouse or one spouse.

UML Notation	Nature of the Relationship	Example		Description
0..*	Zero or many			An employee can have no payroll deductions or many deductions.
0..1	Zero or one			An employee can have no spouse or one spouse.
1	One and only one			An office manager manages one and only one office.
1..*	One or many			One order can include one or many items ordered.

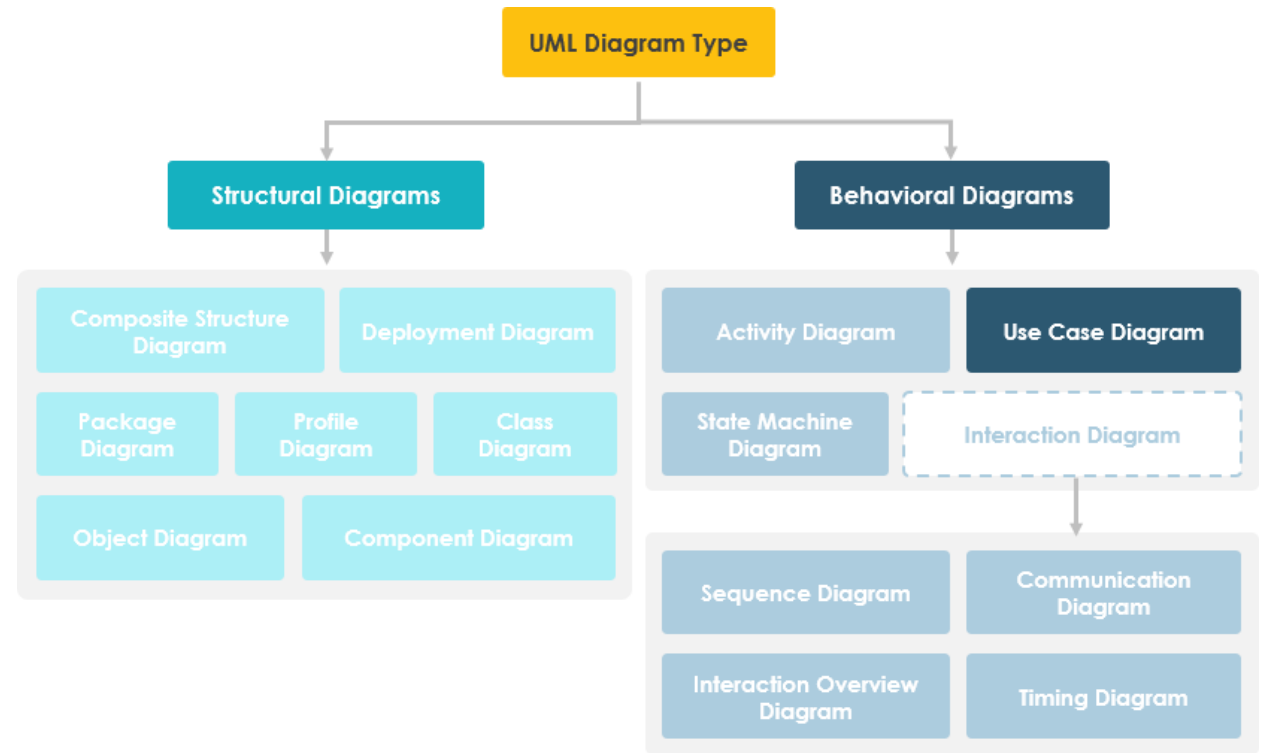
Cardinality

- Class diagram for a sales order use case.
- Notice that the sales office has one sales manager who can have anywhere from zero to many sales reps.
- Each sales rep can have anywhere from zero to many customers, but each customer has only one sales rep.



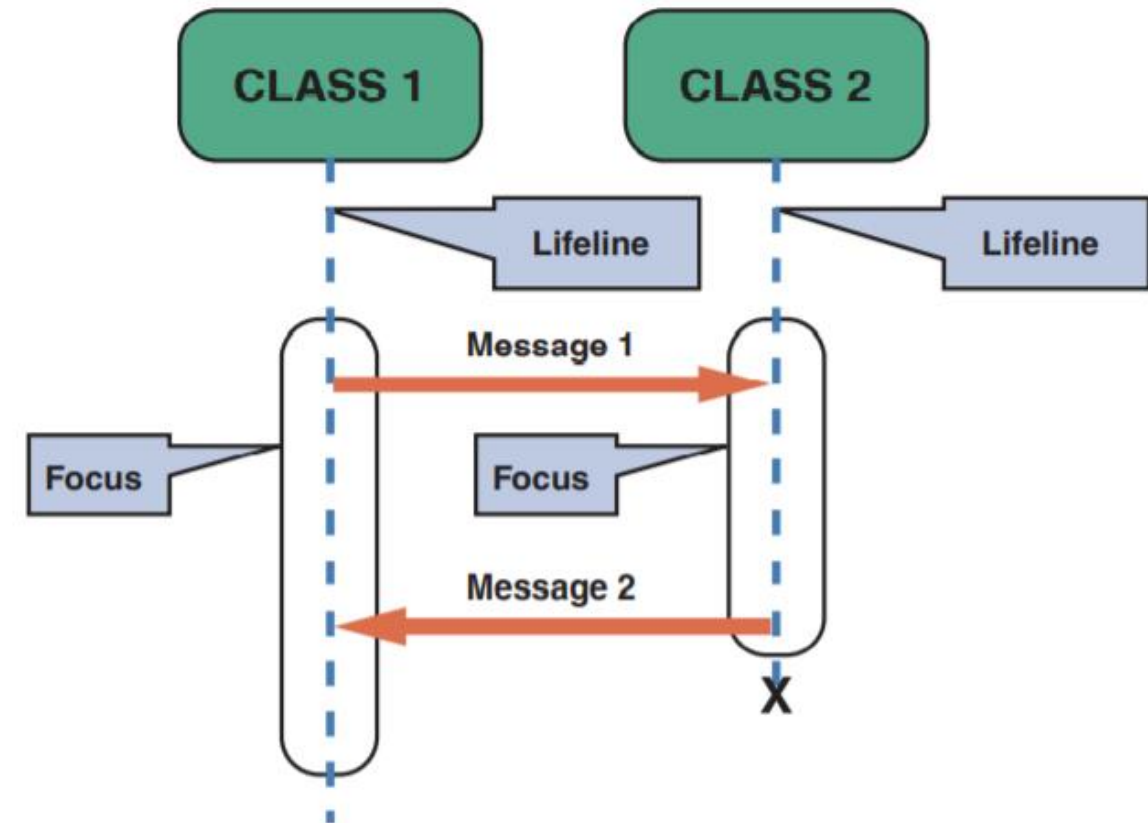
Structural Diagram

- A class diagram is a type of Structural diagram, That can describe the system from its domain.
- Just to map every Data to its operations.



SEQUENCE DIAGRAMS

- A sequence diagram is a dynamic model of a use case, showing the interaction among classes during a specified time period.
- A sequence diagram graphically documents the use case by showing the classes, the messages, and the timing of the messages.
- Sequence diagrams include symbols that represent classes, lifelines, messages, and focuses.



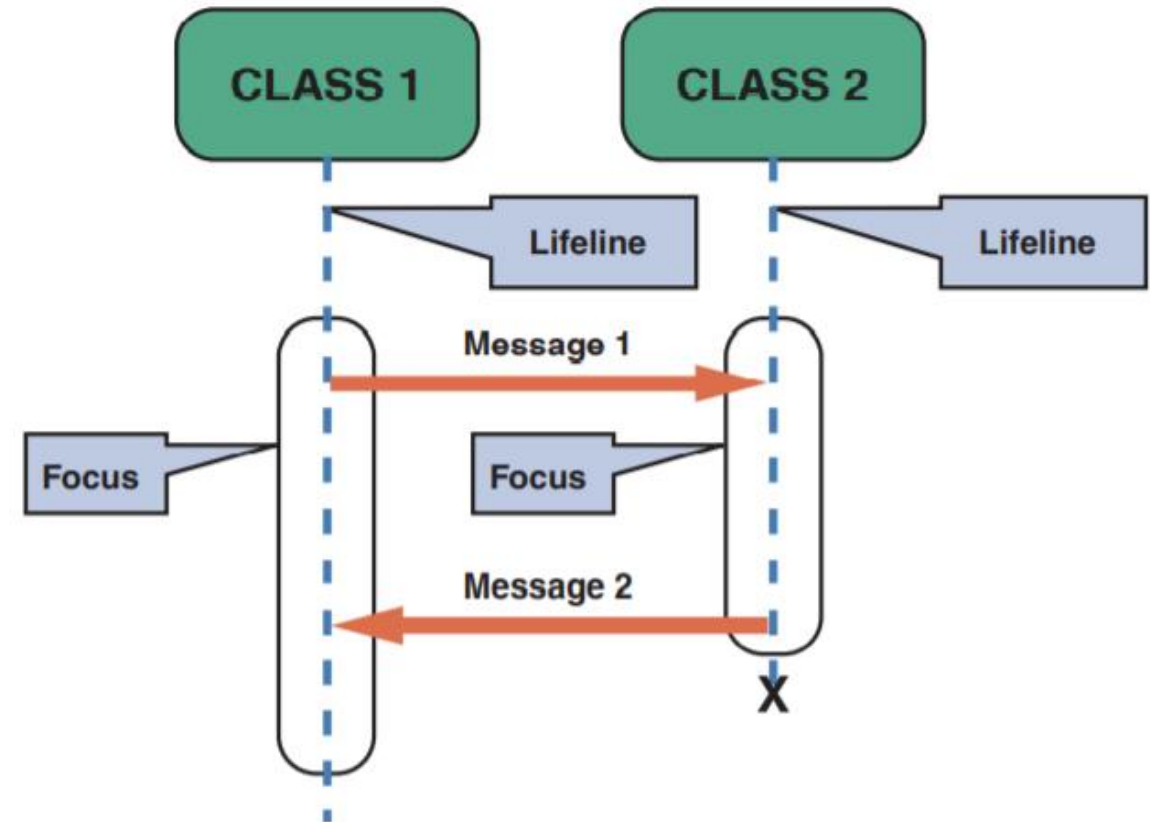
SEQUENCE DIAGRAMS

- **CLASSES**

- A class is identified by a rectangle with the name inside. Classes that send or receive messages are shown at the top of the sequence diagram.

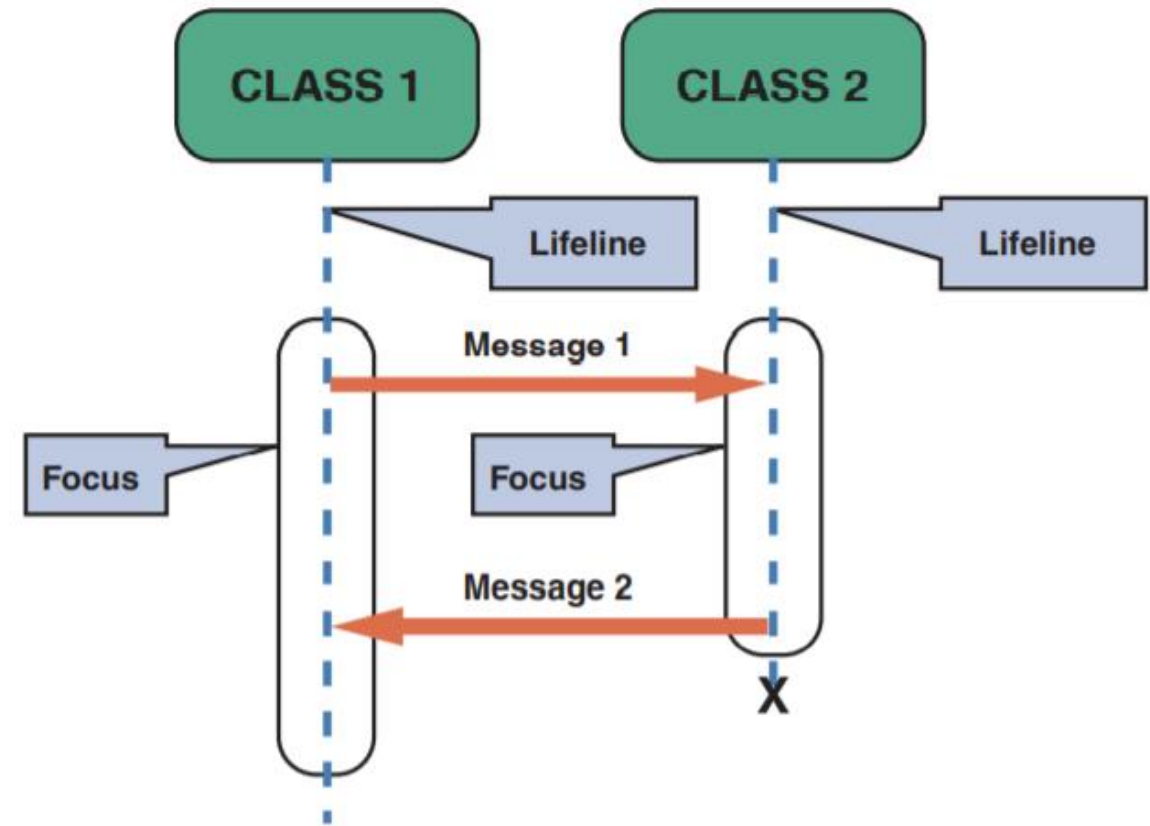
- **LIFELINES**

- A lifeline is identified by a dashed line. The lifeline represents the time during which the object above it is able to interact with the other objects in the use case. An X marks the end of the lifeline.



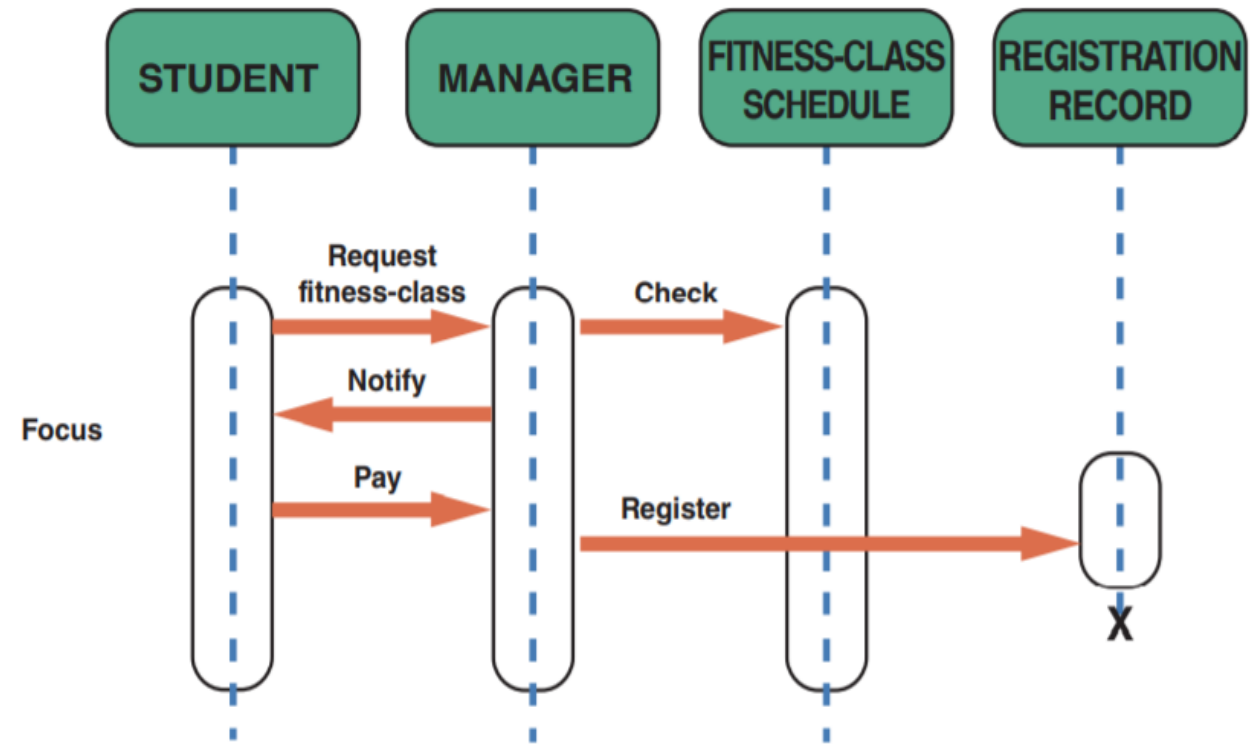
SEQUENCE DIAGRAMS

- **MESSAGES**
- A message is identified by a line showing direction that runs between two objects.
- The label shows the name of the message and can include additional information about the contents.



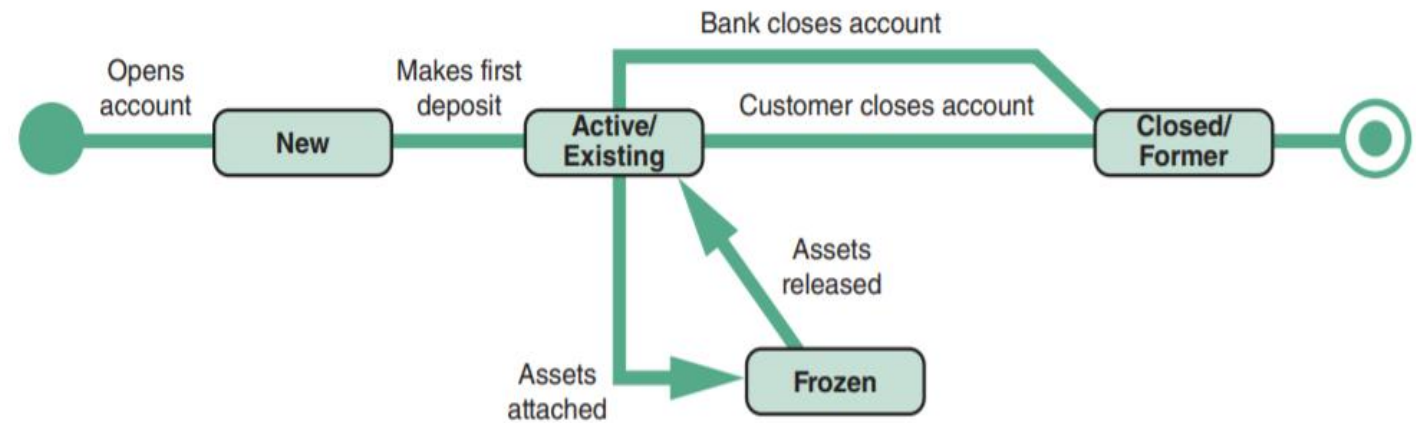
SEQUENCE DIAGRAMS

- Focuses
- A focus is identified by a narrow vertical shape that covers the lifeline.
- The focus indicates when an object sends or receives a message.

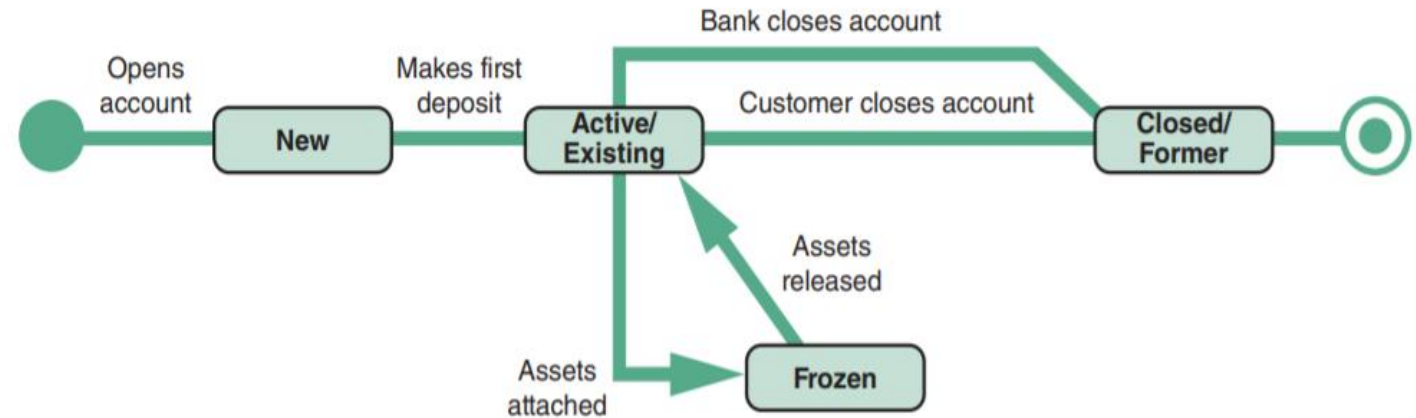


State Transition diagram

- A state transition diagram shows how an object changes from one state to another, depending on events that affect the object.
- All possible states must be documented in the state transition diagram



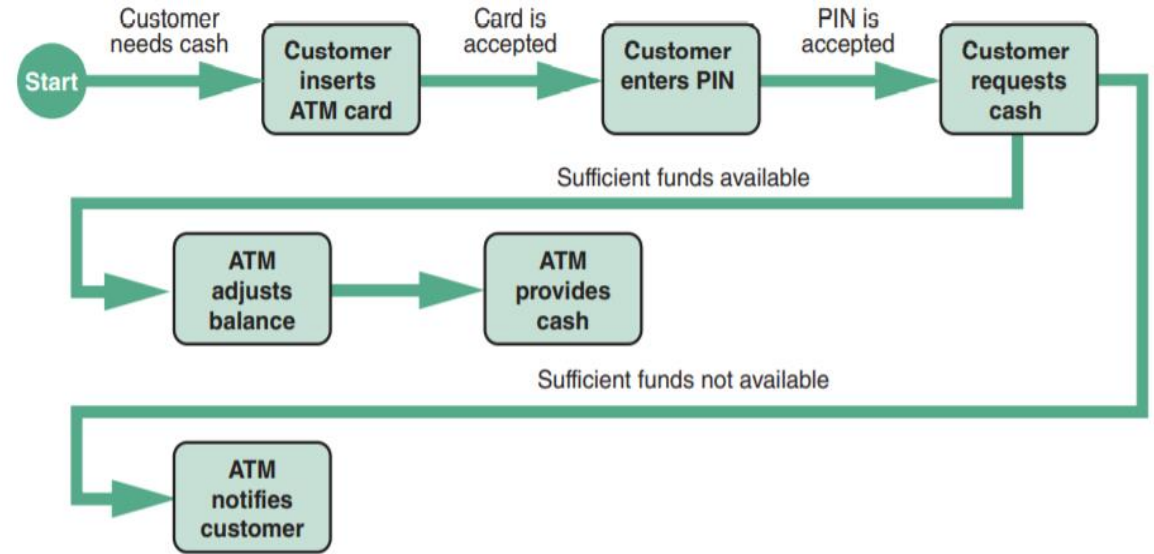
State Transition diagram



- In a state transition diagram, the states appear as rounded rectangles with the state names inside.
- The small circle to the left is the initial state, or the point where the object first interacts with the system.
- Reading from left to right, the lines show direction and describe the action or event that causes a transition from one state to another.
- The circle at the right with a hollow border is the final state.

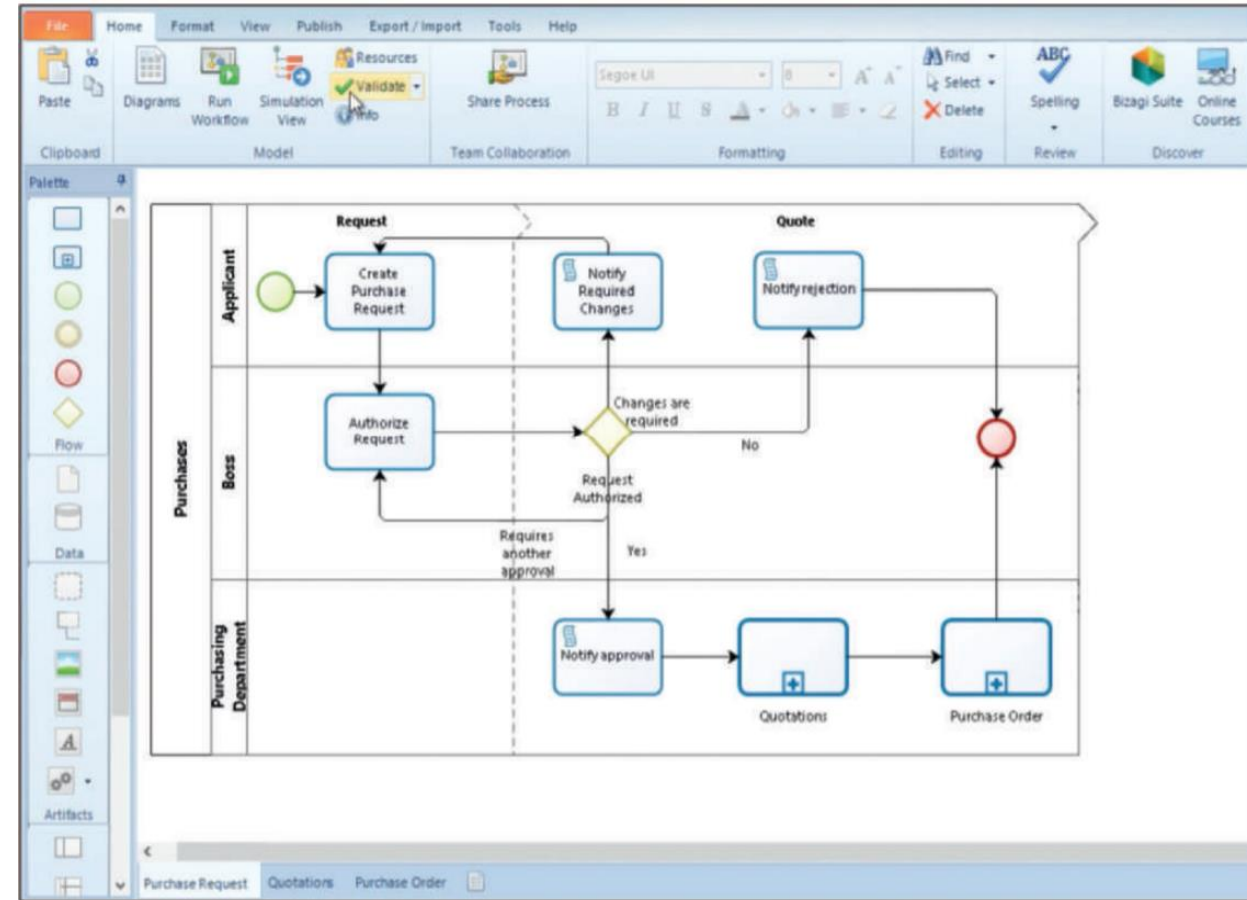
Activity Diagram

- An activity diagram resembles a horizontal flowchart that shows the **actions** and events as they occur.
- Activity diagrams show the order in which the actions take place and identify the outcomes.
- Activity diagrams also can display multiple use cases in the form of a grid, where classes are shown as vertical bars and actions appear as horizontal arrows.



BUSINESS PROCESS MODELING

- In addition to sequence diagrams and activity diagrams, business process models (BPM) can be used to represent the people, events, and interaction in a system.
- BPM initially was discussed in Chapter 4 as a requirements modeling tool, but it can be used anytime during the systems development process. BPM works well with object modeling, because both methods focus on the actors and the way they behave.



CASE Tools



- Object modeling requires many types of diagrams to represent the proposed system.
 - Creating the diagrams by hand is time consuming and tedious, so systems analysts rely on CASE tools to speed up the process and provide an overall framework for documenting the system components.
 - In addition, CASE tools ensure consistency and provide common links so that once objects are described and used in one part of the design, they can be reused multiple times without further effort.
- 



THANK YOU!

